Marketplace E-commerce Website — Developer Documentation

Project: **my-ecommerce-website** · Stack: Next.js (App Router), NextAuth (credentials), Prisma (PostgreSQL), Stripe (Checkout + Connect + Webhooks), next-intl (i18n), Tailwind CSS + Radix UI
Audience: developer handover (implementation notes, lifecycle steps, screenshot frames, and how to demonstrate each feature)

How to save this document as a PDF

Open this file in Chrome or Edge, press `Ctrl` + `P` (Mac: `Cmd` + `P`), choose **Save as PDF** (or **Microsoft Print to PDF**), then save. For the best layout, enable **Background graphics** if your browser offers it.

---

1. What this project is

This is an Amazon-style marketplace web app with a product catalog, cart and checkout, customer accounts, seller onboarding with Stripe Connect, and an admin area for managing products, coupons, orders, and analytics.

2. Project lifecycle: planning → creating → developing → testing

After each step below, capture a screenshot and paste it into the dashed frame. This gives you a complete visual record from idea to verified behavior.

2.1 Planning

Goal: agree on scope, data, payments, and roles before writing code. Deliverables: entity list, main user journeys, and environment checklist.

Step P1: List actors (customer, seller, admin) and primary flows.

- Browse → cart → checkout
- Seller onboarding (Stripe Connect)
- Admin catalog + operations

Figure P1 — Planning artifact

Screenshot area — capture this step from your screen and paste here (or overlay in a PDF editor). Example: whiteboard, Notion page, or simple diagram of user flows.

Step P2: Sketch the data model (align to `prisma/schema.prisma`).

- User + Role (CUSTOMER / SELLER / ADMIN)
- Seller (shop profile + Stripe Connect account id)
- Product, Category, ProductImage
- Cart (guest via cookie sessionId + logged-in via userId)
- Order + OrderLine (split per seller)
- Coupon + CouponRedemption
- Review + WishlistItem
- Subscription + SubscriptionEntitlement
- StripeEvent (webhook idempotency)

Figure P2 — Schema / ER

Screenshot area — capture a screenshot of the Prisma schema in the IDE, or Prisma Studio showing tables after seed.

Step P3: Define payments (Stripe Checkout + Stripe Connect + Webhooks).

- One-time purchases: `POST /api/checkout` → Stripe Checkout (payment mode)
- Subscriptions: `POST /api/checkout/subscription` → Stripe Checkout (subscription)
- Webhook: `POST /api/webhooks/stripe` (Checkout + Subscription lifecycle)
- Test vs live keys in env (never commit real secrets)

Figure P3 — Stripe + env plan

Screenshot area — capture a screenshot of `.env.example` or a checklist table (without real secret values).

Step P4: Plan i18n (supported locales + URL shape under `/[locale]/…`).

- Locales: `en`, `fr` (see `src/i18n/routing.ts`)
- Locale segment layout: `src/app/[locale]/layout.tsx`

Figure P4 — Locale routing

Screenshot area — capture `src/i18n/routing.ts` or the `src/app/[locale]/` folder in the repo.

2.2 Creating (project setup)

Goal: runnable app on your machine with database and seeded data. Deliverables: dependencies installed, DB migrated, dev server up.

Step C1: Install dependencies.

```
npm install
```

Figure C1 — npm install

Terminal showing successful `npm install` at project root.

Step C2: Configure PostgreSQL and create `.env` from `.env.example`; set `DATABASE_URL`.

Figure C2 — Environment file

Editor showing `DATABASE_URL` and placeholders (blur secrets if sharing).

Step C3: Apply schema and seed.

```
npm run db:push
npm run db:seed
```

Figure C3 — DB push and seed

Terminal output ending with `Seed OK` or similar.

Step C4: Start the app and open the locale route.

```
npm run dev
```

Then open `http://localhost:3000/en`.

Figure C4 — App running

Browser showing the marketplace home or products page.

2.3 Developing

Goal: implement or extend features using App Router pages, route handlers under `src/app/api`, and Prisma. Work in small slices: API → UI → webhook if needed.

Step D1: Pick a feature (example: cart) and locate API + UI.

- API route: `src/app/api/cart/route.ts`
- UI: cart page under `src/app/[locale]/cart` (and header cart badge)

Figure D1 — Code navigation

IDE showing the API route + related page component side by side.

Step D2: Confirm API calls in DevTools Network tab.

- Example: `POST /api/cart` returns `200` with JSON `{ ok: true }`

Figure D2 — Network tab

DevTools Network entry for `/api/cart` with 200 response.

Step D3: Stripe flow demo (Checkout redirect).

- Set `STRIPE_SECRET_KEY` and `NEXT_PUBLIC_APP_URL` in `.env`
- Trigger Checkout and confirm redirect to the Stripe-hosted page

Figure D3 — Stripe Checkout

Browser on Stripe test Checkout page after redirect from your app.

2.4 Testing (verify behavior)

Goal: prove each feature works end-to-end. Combine manual UI tests, API checks, database inspection, and Stripe test mode.

Step T1: Smoke test.

- Register, login, browse products
- Add to cart (guest and logged-in)
- Open wishlist and reviews

Figure T1 — Smoke test

Any key screen showing success (e.g. cart with line items).

Step T2: Checkout test.

- Complete payment with Stripe test card `4242 4242 4242 4242`
- Land on `/en/checkout/success`
- See orders under account

Figure T2 — Order confirmation

Checkout success page or account orders list.

Step T3: Webhook test (Stripe CLI) + DB proof.

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

Figure T3 — Webhook or DB proof

Stripe CLI output showing event received, or Prisma Studio Order table.

Step T4: Role test.

- Login as `admin@example.com` → open `/en/admin`
- Login as `seller@example.com` → open `/en/seller`
- Confirm unauthorized users are redirected to login with `callbackUrl`

Figure T4 — Admin or seller dashboard

Protected area visible only with the correct role.

3. Quick start (local dev)

Prereqs: Node.js, npm, PostgreSQL, and (optional) Stripe CLI for webhook forwarding.

```
npm install
copy .env.example .env
npm run db:push
npm run db:seed
npm run dev
```

Optional database UI: `npm run db:studio`

3.1 Seeded demo accounts

Seed password for all accounts: **demo1234**

- `admin@example.com` (ADMIN)
- `customer@example.com` (CUSTOMER)
- `platform@example.com` (SELLER, “Marketplace Official”)
- `seller@example.com` (SELLER, “Partner Goods Co”)

4. System overview (developer)

4.1 Routing + layouts

The UI uses locale-prefixed routing under `/[locale]/…` with next-intl (see `src/app/[locale]/layout.tsx` and `src/i18n/routing.ts`). Admin and Seller areas are protected at the layout level using server-side session checks and redirects to login with `callbackUrl`.

4.2 Authentication + roles

Authentication is NextAuth with a credentials provider (email/password). Passwords are stored as bcrypt hashes in `User.passwordHash`. Sessions use the JWT strategy; the user role is copied into the JWT and then into `session.user.role` (see `src/auth.ts`).

4.3 Data layer

Prisma models cover users/roles, sellers, products/categories/images, carts (guest + user), orders + order lines, reviews, wishlist, coupons/redemptions, subscriptions/entitlements, and `StripeEvent` for webhook idempotency (see `prisma/schema.prisma`).

4.4 Payments

Stripe is used for one-time Checkout sessions and subscription Checkout sessions. A webhook handler finalizes paid checkouts, decrements stock, splits a single checkout

into per-seller orders, records coupon redemption, clears cart, and syncs subscriptions + entitlements.

5. Features: implementation notes + how to demonstrate

5.1 Catalog browsing + product details

What it is: A product catalog with detail pages keyed by slug. Products are stored in PostgreSQL (Prisma) with images and category + seller relations.

Key pieces: product routes under `src/app/[locale]/`; models `Product` + `ProductImage`; seeded products in `prisma/seed.ts`.

How to demonstrate:

- Run seed (`npm run db:seed`) so products exist.
- Open `/en/products` and confirm a product grid renders.
- Open a product detail page `/en/products/<slug>`.
- Optional: reduce `Product.stock` in Prisma Studio and refresh cart behavior.

Figure 5.1 — Catalog and product detail

Paste a screenshot showing the product list and/or a product detail page after the steps above.

5.2 Search

What it is: A search UI route that queries the catalog and displays results.

How to demonstrate:

- Open `/en/search`.
- Search for a seeded keyword (e.g., “Coffee” or “Speaker”).
- Verify results update and link into product details.

Figure 5.2 — Search results

Paste a screenshot of the search page with results.

5.3 Register + login (NextAuth credentials)

What it is: Customer registration via `POST /api/auth/register` and sign-in via NextAuth credentials provider. Roles are enforced across protected areas.

Key pieces:

- `src/app/api/auth/register/route.ts`
- `src/app/api/auth/[...nextauth]/route.ts`
- `src/auth.ts` JWT/session callbacks (role propagation)

How to demonstrate:

- Register a new account from `/en/register` and confirm success.
- Login from `/en/login` and verify session persists on refresh.
- Try `/en/admin` as non-admin; confirm redirect.
- Login as `admin@example.com` and confirm `/en/admin` loads.

Figure 5.3 — Auth and role protection

Paste a screenshot of login success or admin dashboard after login.

5.4 Cart (guest + logged-in) + quantity updates

What it is: A cart that works for guests (HTTP-only cookie sessionId) and signed-in users (user cart). Cart supports add, update quantity, remove (quantity 0), and clear.

Key pieces:

- `src/app/api/cart/route.ts` (POST/PATCH/DELETE)

- `Cart` + `CartItem` models
- Guest session cookie id via `CART_COOKIE` (see `src/lib/cart`)

How to demonstrate:

- Logged out: add a product to cart, open `/en/cart`.
- Change quantity; verify stock validation (try beyond stock).
- Remove a line (set quantity to 0) and verify it disappears.
- Login and add items; confirm the cart works for the user.

Figure 5.4 — Cart

Paste a screenshot of `/en/cart` with line items.

5.5 Checkout (Stripe one-time payments) + success flow

What it is: A server endpoint creates a Stripe Checkout session in payment mode for the logged-in user's cart. On completion, the webhook fulfills the order, decrements stock, splits orders per seller, and clears the cart.

Key pieces:

- `POST /api/checkout` (creates Checkout session)
- `POST /api/webhooks/stripe` (`checkout.session.completed`)
- `Order` / `OrderLine` tables (split per seller)
- `StripeEvent` idempotency table

How to demonstrate:

- Set `STRIPE_SECRET_KEY` + `NEXT_PUBLIC_APP_URL` in `.env`.
- Login as `customer@example.com` and add 1–2 products to cart.
- Go to `/en/checkout`, enter an address, start checkout (redirect to Stripe).
- Pay with Stripe test card `4242…`.
- Verify redirect to `/en/checkout/success`, then check `/en/account/orders` for created orders.
- Verify cart cleared and product stock decreased (Prisma Studio).

Figure 5.5 — Checkout + success

Paste Stripe Checkout and/or checkout success and order history.

5.6 Coupons (validate + apply discount)

What it is: Coupon validation computes discounts based on cart subtotal and coupon rules. Checkout can create a one-time Stripe coupon to apply discounts to the session.

Key pieces:

- `Coupon` + `CouponRedemption` models
- `GET /api/coupon?code=…`
- `computeCouponDiscount` used by `POST /api/checkout`

How to demonstrate:

- Login and add items so subtotal meets coupon minimum.
- Call `/api/coupon?code=SAVE10` and verify ok response.
- Proceed to checkout with a coupon code and verify reduced amount on Stripe Checkout.
- After payment, confirm a `CouponRedemption` record exists (Prisma Studio).

Figure 5.6 — Coupon applied

Paste screenshot of discounted total on Stripe Checkout or coupon UI.

5.7 Wishlist

What it is: Per-user wishlist items with add/remove and a check endpoint.

Key pieces:

- `WishlistItem` model

- `/api/wishlist` GET/POST/DELETE

How to demonstrate:

- Login, open a product page, add it to wishlist.
- Open `/en/wishlist` and confirm the product appears.
- Remove it and verify it disappears; confirm API reflects `onList: false`.

Figure 5.7 — Wishlist

Paste screenshot of `/en/wishlist` with items.

5.8 Reviews (including verified purchase behavior)

What it is: Users can upsert one review per product. Reviews are marked `verifiedPurchase` if the user has purchased the product.

Key pieces:

- `Review` model unique(`userId` , `productId`)
- `/api/reviews` GET/POST
- Verified purchase check via `OrderLine` join in the API

How to demonstrate:

- As a logged-in user who has NOT purchased a product, submit a review; verify it saves.
- Purchase that product, then update the review.
- Verify `verifiedPurchase` becomes true (DB inspection).

Figure 5.8 — Reviews

Paste screenshot of reviews on product page or review form.

5.9 Subscriptions (subscription checkout + webhook sync)

What it is: Subscription-eligible products can be purchased through Stripe Checkout subscription mode using a Stripe price id. Webhooks upsert Subscription rows and manage entitlements (`subscription_active`).

Key pieces:

- `Product.isSubscriptionEligible` + `Product.stripePriceId`
- `POST /api/checkout/subscription`
- `POST /api/webhooks/stripe` subscription events
- `Subscription` + `SubscriptionEntitlement` tables

How to demonstrate:

- Create a recurring Stripe Price in Stripe dashboard.
- Set that id on a subscription-eligible product (`stripePriceId`).
- Login and start subscription checkout.
- Verify Subscription row + entitlement key `subscription_active` in DB.
- Cancel in Stripe test dashboard and verify entitlement removal after webhook processing.

Figure 5.9 — Subscription

Paste Stripe subscription checkout or `/en/subscriptions` page.

5.10 Seller area (dashboard, products, orders)

What it is: Seller pages for managing seller-related views. Access is restricted to SELLER or ADMIN in the seller layout.

Key pieces:

- `src/app/[locale]/seller/*` pages
- `src/app/[locale]/seller/layout.tsx` role guard

How to demonstrate:

- Login as `seller@example.com` and open `/en/seller`.
- Navigate to `/en/seller/products` and `/en/seller/orders`.
- Logged out: open `/en/seller` and confirm redirect to login.

Figure 5.10 — Seller area

Paste screenshot of seller dashboard or orders.

5.11 Stripe Connect onboarding for sellers

What it is: Sellers can create or reuse a Stripe Express account and generate an onboarding link. Seller record stores `stripeConnectAccountId`. For single-seller carts, the platform can take an application fee and transfer funds to the connected account.

Key pieces:

- `POST /api/seller/connect`
- `Seller.stripeConnectAccountId`
- Connect fee logic in `POST /api/checkout` (single-seller carts only)

How to demonstrate:

- Set `STRIPE_SECRET_KEY` in `.env`.
- Login as `seller@example.com` and open `/en/seller/onboarding`.
- Start onboarding and complete Express flow in test mode.
- Confirm `Seller.stripeConnectAccountId` saved in DB.

Figure 5.11 — Stripe Connect onboarding

Paste Stripe Express onboarding or return URL after completion.

5.12 Admin area (products, coupons, orders, analytics)

What it is: Admin dashboard for operational control. Access is restricted to ADMIN at the admin layout level.

Key pieces:

- `src/app/[locale]/admin/*` pages
- `src/app/[locale]/admin/layout.tsx` role guard

How to demonstrate:

- Login as `admin@example.com` and open `/en/admin`.
- Open `/en/admin/products` and create a new product.
- Edit an existing product and confirm changes persist.
- Open `/en/admin/coupons` and verify seeded coupons exist (SAVE10, FLAT500).
- Open `/en/admin/orders` after a checkout and confirm orders appear.

Figure 5.12 — Admin

Paste screenshot of admin products list or orders.

5.13 Product image upload + Blob media streaming

What it is: An authenticated endpoint accepts multipart uploads and stores images either locally (`public/uploads`) or in Vercel Blob (optional). If Blob is private, images can be streamed via `/api/media/blob` with pathname restrictions.

Key pieces:

- `POST /api/upload/product-image`
- `GET /api/media/blob?pathname=…`
- `BLOB_READ_WRITE_TOKEN` and optional `BLOB_ACCESS` env vars

How to demonstrate:

- Login as ADMIN or SELLER.
- Upload a product image via the product editor UI.
- Verify returned URL is stored in `ProductImage.url` and renders on product page.
- If using private Blob: request `/api/media/blob?pathname=products/...` and confirm it streams with caching headers.

Figure 5.13 — Product image

Paste product page showing uploaded image or upload UI.

5.14 Multi-language / locale routing (next-intl)

What it is: Locale segment routing under `/[locale]/…` with server-side message loading. Only supported locales are allowed; others return `notFound()`.

Key pieces:

- `src/app/[locale]/layout.tsx`
- `src/i18n/routing.ts` (`locales: ["en", "fr"]`)

How to demonstrate:

- Open `/en/products` then `/fr/products`.
- Verify locale persists in navigation links.
- Try an unsupported locale (e.g., `/es/products`) and confirm notFound behavior.

Figure 5.14 — Locales

Paste two screenshots of en vs fr pages.

5.15 Account export (download JSON)

What it is: An authenticated endpoint that returns a JSON export of user data (profile fields, orders+lines, addresses, reviews, wishlist).

Key pieces:

- `GET /api/account/export`
- `Content-Disposition` header triggers download

How to demonstrate:

- Login as a customer and place an order and/or create reviews/wishlist items.
- Open `/api/account/export` in the browser.
- Confirm it downloads as `export-<userId>.json` and includes expected objects.

Figure 5.15 — Account export

Paste browser download bar or JSON opened in editor (redact if needed).

6. Main pages / routes (UI)

All UI routes below are under `/[locale]/…` (example: `/en/products`).

- Home: `/[locale]/`
- Products: `/[locale]/products`
- Product details: `/[locale]/products/[slug]`
- Search: `/[locale]/search`
- Cart: `/[locale]/cart`
- Checkout: `/[locale]/checkout`
- Checkout success: `/[locale]/checkout/success`
- Wishlist: `/[locale]/wishlist`
- Subscriptions: `/[locale]/subscriptions`
- Auth: `/[locale]/login`, `/[locale]/register`
- Account orders: `/[locale]/account/orders`
- Seller: `/[locale]/seller`, `/[locale]/seller/onboarding`, `/[locale]/seller/products`, `/[locale]/seller/orders`
- Admin: `/[locale]/admin`, `/[locale]/admin/products`, `/[locale]/admin/products/new`, `/[locale]/admin/products/[id]`, `/[locale]/admin/orders`, `/[locale]/admin/coupons`, `/[locale]/admin/analytics`
- Policies: `/[locale]/policies/privacy`, `/[locale]/policies/seller-terms`

7. API endpoints (server routes)

The app uses Next.js route handlers under `src/app/api` (mounted at `/api`).

- Auth: `POST /api/auth/register`, `GET/POST /api/auth/[...nextauth]`
- Cart: `POST /api/cart`, `PATCH /api/cart`, `DELETE /api/cart`
- Wishlist: `GET/POST/DELETE /api/wishlist`
- Reviews: `GET/POST /api/reviews`
- Coupons: `GET /api/coupon?code=…`
- Checkout (one-time): `POST /api/checkout`
- Checkout (subscription): `POST /api/checkout/subscription`
- Stripe webhooks: `POST /api/webhooks/stripe`
- Seller onboarding: `POST /api/seller/connect`
- Upload product image: `POST /api/upload/product-image`
- Private Blob streaming: `GET /api/media/blob?pathname=…`
- Account export: `GET /api/account/export`

8. Data model (Prisma / PostgreSQL)

Core entities:

- Users with roles: CUSTOMER, SELLER, ADMIN
- Sellers (shop profile + Stripe Connect account id + verification status)
- Products, categories, product images
- Cart + cart items (guest via sessionId, logged-in via userId)
- Orders + order lines (orders stored per-seller; checkout can split a single paid session into multiple seller orders)
- Reviews (one review per user+product), verifiedPurchase flag
- Wishlist items
- Coupons + redemptions
- Subscriptions + subscription entitlements (key: subscription_active)
- StripeEvent table for webhook idempotency

9. Authentication and authorization

Authentication is handled by NextAuth using a credentials provider (email + password hash). Sessions are JWT-based. Role-based access is enforced in server layouts for `/admin` and `/seller`.

10. Payments and Stripe

10.1 One-time checkout

A logged-in user creates a Stripe Checkout session for cart items. The checkout payload includes shipping, optional estimated tax (when Stripe Tax is off), and optional coupons. If the cart contains items from a single seller with Stripe Connect configured, the payment intent includes an application fee and transfer destination.

10.2 Webhooks

Stripe webhooks finalize orders, decrement stock, split orders per seller, record coupon redemption, clear cart, and sync subscription lifecycle events. Idempotency is enforced via the `StripeEvent` table.

11. Environment variables

Create a `.env` file based on `.env.example`. Key variables:

- `DATABASE_URL`: PostgreSQL connection string
- `NEXTAUTH_SECRET`, `NEXTAUTH_URL`
- `NEXT_PUBLIC_APP_URL`: public base URL for Stripe redirects
- `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`
- `STRIPE_ENABLE_TAX`, `TAX_BPS` (optional estimated tax)
- `BLOB_READ_WRITE_TOKEN`, `BLOB_ACCESS` (optional Vercel Blob)

12. Notes and constraints

- Currency amounts are stored as integers (cents) in the database and Stripe line items.
- Cart supports guest sessions via an HTTP-only cookie.
- Reviews are limited to one per user per product (upsert).
- Webhook handler uses a `StripeEvent` table for idempotency.

End of document.